

OpenJev: Typed Decisions from Small Models

What it takes to make a 150M encoder, or a 1.7B decoder,
answer many typed questions about one input in a single forward pass

Prathamesh Saraf

github.com/S1LV3RJ1NX/openjev

September 2026

Abstract

Many production decisions are not open-ended generation but a fixed menu: route this message to one of 77 intents, flag whether it mentions clinical risk, rate its urgency. Jev, a commercial API from TypeSafe, makes exactly this the primitive, and does it well: it answers many typed questions about one input in a single call, and on a 77-way intent task it has never seen it scores roughly 0.820. This paper asks what it takes to build an open equivalent, and reports honestly where that succeeds and where it does not.

What we built. OpenJev encodes the state once as a shared prefix, places every candidate option after it behind a marker token, isolates questions from one another with block-diagonal attention, and normalises scores within each question’s own option set. Ten questions therefore cost one forward pass and 6% more latency than one, and the output is a calibrated probability over a closed menu rather than generated text. We release the code, three checkpoints, and three datasets including a 279-task, 323,466-row training mixture.

The main finding is negative, and it is the useful one. The architecture does not generalize on its own. A strong bidirectional encoder with this exact output format, untrained, scores $0.7\times$ **chance** on held-out schemas, and a 61-task training mixture leaves it at $0.9\times$. Generalization is instead a property of the training data. Padding training menus with borrowed distractor labels, together with a mixture rebuild in the same change, moves a 77-way held-out menu from a literal 0.000 to 0.205; a model that has never seen a large menu collapses onto one label when handed one, though we did not separate the two interventions and so cannot attribute the gain cleanly. Scaling and auditing the mixture to 279 tasks reaches $21.9\times$ **chance**, with all seven held-out tasks clearing their floor and a 151-way menu at $94.9\times$.

Against Jev, the result splits cleanly on one axis: labels. Given 395 labelled examples and 258 seconds, a rank-16 LoRA adapter on Qwen3-1.7B scores intent 0.979 against Jev’s 0.941 ($p = 1e-3$) and multi-label exact set 0.909 against 0.822 ($p = 7e-6$) on a hardened healthcare routing benchmark: three wins, five statistical ties, no losses. **The comparison is asymmetric and in our favour**, since we are fine-tuned on that task and Jev is zero-shot on it, so the claim it supports is that a few hundred labels are worth more than the gap between an open 1.7B model and a closed API. The adapter is 87 MB and outperforms opening all 1,725M parameters (0.929, 3.4 GB). Without labels the picture inverts: 0.290 on Banking77 against Jev’s 0.820, and applied zero-shot to the routing task our general checkpoint reaches intent 0.601 with a clinical-gate recall of 0.111, which is not a usable gate.

Takeaway. Use Jev, or something like it, when you need typed decisions on a schema you have no labels for; that capability is real and we did not reproduce it. Use OpenJev when you can label a few hundred examples of your actual task, want the model on your own hardware, need full-precision probabilities and determinism, or need multi-label routing where a single choice cannot express the answer. Benchmark transfer and deployment readiness are not the same thing, and we report the gap between them rather than the better number. All claims are reproducible from a public repository, with negative and retracted results given the same prominence as the positive ones.

1 Introduction

Consider a pharmacy support queue. A message arrives: *“I need a refill on my thyroid tablets, and are you open on Sunday? Also I’ve been getting dizzy since starting them.”* A system routing this must decide, simultaneously, which topics it concerns (two), whether it mentions clinical risk that a human should see (it does), and how urgent it is. Each of these is a decision over a *known, closed* set of answers.

The dominant approach is to ask a language model and parse what comes back. This works, and it is expensive in three distinct ways. Each question costs its own generation. The output is unconstrained, so it must be coerced onto the menu by grammar-constrained decoding or by retrying. And the resulting token probabilities are a poor basis for the thresholding that operational systems need, because they describe a sampling distribution over strings rather than a calibrated belief over options.

Readers new to this area: Appendix A defines every technical term used here, from “token” through to the statistical tests, so the main text can stay short.

1.1 Typed decisions

We use *typed decision* for a question whose answer set is fixed in advance and known to the system. Three types cover most of what production routing and guardrail systems need:

- **choice:** exactly one of K options. Intent, category, routing target. K reaches 151 in our evaluation.
- **score:** one level on an *ordered* scale. Severity, urgency, quality. The ordering is information a **choice** discards.
- **noul:** yes or no. Flags and safety gates. Several asked together express multi-label structure, which a single **choice** cannot: a softmax over intents has no way to say “both”.

1.2 Jev, and why it is the right thing to measure against

The system that made this concrete for us is **Jev**, a commercial typed-decision API from TypeSafe. It is closed, and it is good. Rather than treat it as a competitor to be beaten on a leaderboard, we treat it as an existence proof and a measuring stick, and it deserves credit on both counts.

Three things Jev gets right, which we verified by measurement rather than taking on faith:

- **It asks the right question.** Most of the industry responds to structured-output needs by constraining a generative decoder. Jev instead makes the typed decision the primitive. The API accepts a state and a set of typed questions and returns probabilities over fixed menus. Having used it, we find this to be the correct abstraction for routing and guardrail work, and the framing of this paper is borrowed from it.

- **The shared-prefix design pays off, and we can see it in the timings.** Latency is flat from 2 to 255 options, and an eight-question call costs the same wall time as a one-question call (392 ms against 407 ms). Adding 253 options costs roughly 8,200 input tokens and approximately zero milliseconds. That is a real engineering result and it is what convinced us the architecture in §3 was worth building.
- **Its zero-shot ability is the genuinely hard part, and it is far ahead of ours.** On Banking77, a 77-way intent task it has never been trained on by us, it scores roughly 0.820. Our best equivalent is 0.290. After considerable effort we did not close this gap, and §6.6 shows the gap is wider still on a realistic deployment task. If you need typed decisions on a schema you have no labels for, Jev is the better tool and this paper does not argue otherwise.

We also record where we found its limits, because a measuring stick should be described accurately in both directions: probabilities are quantised to a 0.01 grid with 71.9% of returned values at a hard zero, which removes log-odds and tail-risk reasoning; there is no seed, so results are not reproducible run to run; and on our hardened routing benchmark it misses roughly one in five obliquely-worded clinical risks.

1.3 What this paper is about

OpenJev is an open implementation of an architecture for answering many typed questions about one input at once, and an empirical account of what makes it work. We reverse-engineered the observable behaviour of Jev, built an architecture consistent with it, and then measured which parts of the recipe actually carry the performance. The interesting result is not the architecture, which is a recombination of published ideas (§2). It is that almost none of the performance comes from it.

Three findings organise the paper.

The architecture does not generalize on its own. A pretrained bidirectional encoder, given the option-scoring output format and no task training, scores $0.7\times$ chance on held-out schemas, worse than guessing. Its larger sibling reaches $2.2\times$. The output format makes zero-shot scoring *possible*; it does not make it *present* (§6.1).

Generalization is a property of the training mixture, and some of it is synthesizable. Our first mixture left held-out transfer at $0.9\times$ chance. The single largest fix was not more data but a different *shape* of data: padding training menus with distractor labels borrowed from other tasks, which moved a 77-way held-out menu from a literal 0.000 to 0.205. A model that has never seen a large menu collapses onto one label when handed one, and you can manufacture large menus without finding datasets that have them (§6.3).

With labels, small models win; without them, they do not. A rank-16 adapter on a 1.7B backbone, trained on 395 examples, beats the commercial API on three of seven router measures and ties the rest. The same architecture without task labels remains far behind it. That split is the honest summary of where this technology stands (§6.4, §8).

1.4 Contributions

1. An architecture and open implementation for multi-question typed decisions in a single forward pass, with latency flat in the number of questions (10 questions cost 6% more than 1) and full-precision calibrated probabilities.

2. *Label-space augmentation*, a training-time intervention that manufactures the menu-size diversity a mixture lacks, and which is the difference between no transfer and useful transfer at high cardinality.
3. A measured account of parameter-efficient fine-tuning for this task shape, including the result that a 17M-parameter adapter outperforms opening all 1,725M parameters, at 1/39 the artifact size.
4. A reproducible benchmark suite with an enforced contamination guard, a held-in control that distinguishes “no transfer” from “a bug”, and a hardened routing task built to retain headroom where the reference system saturated.
5. A set of negative and retracted results reported at the same prominence as the positive ones (§7), including two interventions that made things worse and one conclusion we withdrew after finding the bug that produced it.

Everything is reproducible from a public repository, with released checkpoints, adapters, and datasets listed in Appendix B.

2 Background and related work

Every structural idea in OpenJev exists in the literature. We state the provenance plainly, because the paper’s contribution is empirical rather than architectural.

Scoring options instead of generating them. UniMC [11] recasts multiple-choice NLU by inserting an option-marker token before each candidate and reading a decision from the encoder’s state at that position, avoiding generation entirely. Our readout is the same idea. The consequence people usually reach for, that such a model “cannot hallucinate”, is narrower than it sounds: the model cannot emit a string outside the menu, which says nothing about whether the option it picks is right.

Encoding a shared prefix once. Parallel Context Windows [5] and related work formalise attention masks in which several segments each attend to a common prefix and to themselves, but not to one another. We use exactly this to isolate questions, so that an answer does not depend on which other questions were asked in the same call. On a causal backbone the prefix is ordinary left-context, which is what a KV cache is for.

Multi-task training for zero-shot transfer. T0 [6] and the Flan collection [3] establish that training across many tasks produces zero-shot ability on unseen ones, and that the gain continues accruing with task count into the high hundreds. Our mixture is assembled from **tasksource** [8], which normalises a large number of classification and multiple-choice datasets into a common interface. Flan’s task-count finding is the reason we treat 279 tasks as a floor rather than a target.

Intermediate-task transfer. Training on an intermediate task before the target is STILTs [4]; the domain-adaptive variant is Gururangan et al. [1]. Both report that the gain is largest when the target dataset is small, and that it is inconsistent across task pairs and can be negative. We observe both effects: a large gain at 395 target examples (§6.4), and two mixture changes that made held-out transfer *worse* (§7).

block 0: shared state	block 1	block 2	block 3
“I need a refill, and are you open Sunday?”	intent? refill • hours •	clinical? no • yes •	urgent? low • high •

Figure 1: Packing. The state is encoded once. Each option ends in a marker • where the score is read. Every block attends to block 0 and to itself; no block attends to another. Asking question 3 therefore cannot change the answer to question 1.

Parameter-efficient adaptation. LoRA [2] trains low-rank updates beside frozen weights. Our use is conventional; the finding that it *outperforms* full fine-tuning here, rather than merely approximating it cheaply, is reported in §6.4 together with the most likely explanation.

Ordinal and rejection heads. CORN/CORAL [7] give rank-consistent ordinal objectives, and adaptive decision boundaries [12] detect out-of-scope inputs without negative samples. We implement neither; **score** is currently a softmax over levels whose order is used only in rendering and in the expectation-based readout. This is a limitation and we mark it as such (§8).

The system we compare against. Our reference is a commercial typed-decision API, `jev-1.13.0` from TypeSafe, measured by black-box probing on 20 September 2026 at the sample sizes stated. We inferred its design from behaviour: latency is flat from 2 to 255 options and an eight-question call costs the same wall time as a one-question call, which is the signature of a shared prefix with parallel readout. All figures attributed to it are one-day measurements of a hosted endpoint from one network location, and should be reproduced before being relied on.

3 Architecture

3.1 Problem statement

An instance is a state s and a set of questions $Q = \{q_1, \dots, q_m\}$. Each question q_i carries instructions and an option set $O_i = \{o_{i,1}, \dots, o_{i,K_i}\}$, with K_i varying freely across questions in the same call. The model must return, for every question, a distribution over that question’s own options:

$$p(o_{i,j} \mid s, q_i), \quad \sum_{j=1}^{K_i} p(o_{i,j} \mid s, q_i) = 1.$$

Two properties are required. The cost must be sublinear in m , since a production router asks ten questions where a benchmark asks one. And $p(\cdot)$ must be a usable probability, because the operational decision is a threshold on it.

3.2 Packing

We write the state once, then each question and its options after it, as a single token sequence (Figure 1); which positions may then attend to which is Figure 2. Each option is followed (or preceded, depending on backbone) by a *marker* token, and the model’s output for that option is read at the marker position.

Formally, the packed sequence carries a block assignment $b(t) \in \{0, 1, \dots, m\}$ for each token position t , where block 0 is the shared state prefix and block i holds question q_i with its options.

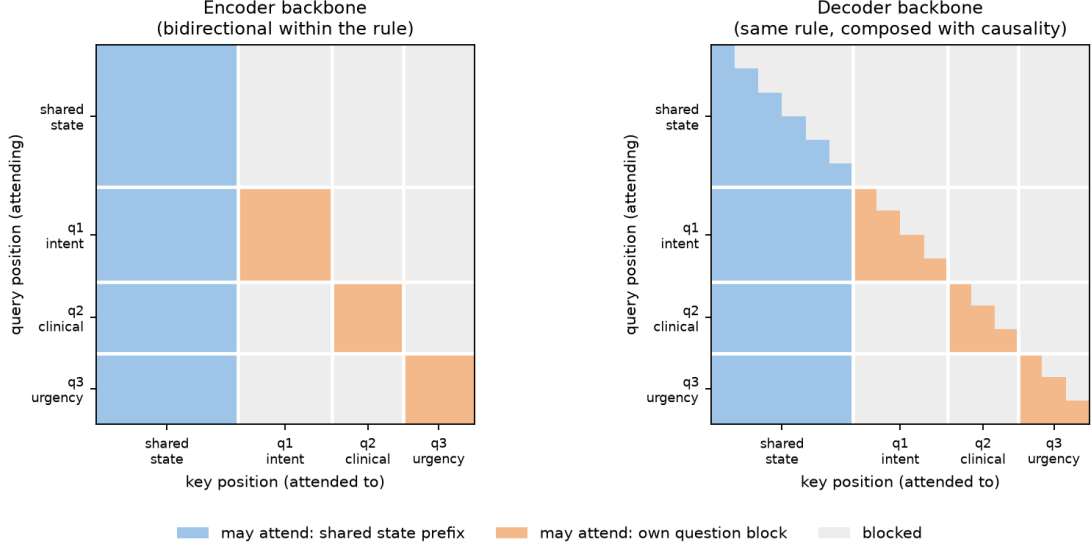


Figure 2: The mask of Eq. 1 for a packed sequence with a shared state and three questions. Blue is attention to the shared prefix, orange is attention within a question’s own block, grey is blocked. No question can see another, so its answer does not depend on what else was asked. Right: the same rule composed with causality on a decoder backbone.

3.3 Block-diagonal attention

Let $A(t, u) \in \{0, 1\}$ indicate whether query position t may attend to key position u . We impose

$$A(t, u) = \underbrace{\mathbb{K}[b(u) = 0 \vee b(u) = b(t)]}_{\text{our rule}} \cdot \underbrace{C(t, u)}_{\text{backbone's own}}, \quad (1)$$

where $C(t, u) = 1$ for a bidirectional encoder and $C(t, u) = \mathbb{K}[u \leq t]$ for a causal decoder. Both factors are indicators, so the product is the conjunction.

The first factor is what makes the shared prefix legitimate: every question sees the full state. The second is what makes the result well defined. Without it, question 2 attends to question 1 and an answer becomes a function of which other questions happened to be asked in the same call. On a causal backbone the state already precedes every question, so the rule’s only remaining work is isolating question blocks from each other. Figure 2 shows both cases.

Memory. The mask is $O(n^2)$ in sequence length, which is the usual attention cost and not specific to this design, but it does bound how many questions fit in one call. At our 2048-token budget a ten-question router with 24 options packs comfortably; a 151-option menu rendered with long descriptions does not, and the implementation degrades by subsampling the menu (always keeping the gold) rather than failing.

3.4 Readout and grouped normalisation

Let h_t be the final hidden state at marker position t , and let $\mathcal{M}_i$ be the marker positions of question q_i . A scalar score is produced per option. For an encoder backbone this is a shared linear scorer, $z_t = w^\top h_t + b$. For a decoder backbone it is the difference of two vocabulary logits at the marker,

$$z_t = [W_{\text{lm}} h_t]_{\text{yes}} - [W_{\text{lm}} h_t]_{\text{no}}, \quad (2)$$

Algorithm 1 Forward pass for one packed instance

Require: state s , questions $Q = \{q_i\}$ with option sets O_i

- 1: $x, b, \mathcal{M} \leftarrow \text{PACK}(s, Q)$ $\triangleright$ tokens, block ids, marker positions
 - 2: $A \leftarrow \text{mask from Eq. 1 using } b \text{ and backbone constraint } C$
 - 3: $H \leftarrow \text{BACKBONE}(x, A)$ $\triangleright$ one forward pass, all questions
 - 4: $z \leftarrow \text{SCORE}(H[\mathcal{M}])$ $\triangleright$ linear, or Eq. 2 for a decoder
 - 5: **for all** questions q_i **do**
 - 6: $p_i \leftarrow \text{SOFTMAX}(z[\mathcal{M}_i])$ $\triangleright$ grouped, Eq. 3
 - 7: **end for**
 - 8: **return** $\{p_i\}$
-

optionally plus a residual head $g(h_t)$ initialised to zero, so that training begins at exactly the untrained readout.

Because K_i differs across questions packed into one sequence, scores are normalised *within* each question rather than globally:

$$p(o_{i,j} \mid s, q_i) = \frac{\exp(z_{t_{i,j}})}{\sum_{t \in \mathcal{M}_i} \exp(z_t)}. \quad (3)$$

This is a segmented softmax over a ragged partition of the flat marker array. Writing y_i for the index of the gold option of question q_i , and $\mathcal{A}$ for the questions this example actually answers (a row may supervise only some of them), the loss is

$$\mathcal{L} = -\frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \log p(o_{i,y_i} \mid s, q_i). \quad (4)$$

A worked example. Suppose one packed sequence holds q_1 with $K_1=3$ and q_2 with $K_2=2$, so the flat marker array has five entries with scores $z = [2.0, 1.0, 0.5, 0.2, 1.4]$ and the partition $\mathcal{M}_1 = \{1, 2, 3\}$, $\mathcal{M}_2 = \{4, 5\}$. A global softmax over all five would make q_1 's options compete with q_2 's, which is meaningless. Grouped, we get $p_1 \approx [0.63, 0.23, 0.14]$ and $p_2 \approx [0.23, 0.77]$, each summing to one. If the golds are $y_1=1$ and $y_2=2$, then $\mathcal{L} = -\frac{1}{2}(\log 0.63 + \log 0.77) \approx 0.36$.

3.5 Rendering the three primitives

How an option is written to the sequence matters more than one would expect, and §6.1 quantifies it. The rules we converged on:

- **choice:** label and description both, since both carry meaning. Where the two are identical, as in multiple-choice tasks whose option *is* its own description, they are collapsed, or every option is emitted twice.
- **score:** the level description alone. Rendering a level as "0: **very negative:** ..." asks the model to score a bare index.
- **noul:** bare polarity, *no/yes*, and *not* the descriptions. A **noul**'s two descriptions necessarily restate one judgement from opposite sides, so scoring both measures their overlap. Measured, this costs AUROC 0.63 against 0.712 (§6.1).

3.6 Cost

The state is encoded once regardless of m , so the marginal cost of a question is its own options. Measured on the ten-question router with a 1.7B backbone, median latency per state is 69ms for one question and 74ms for ten: **ten questions cost 6% more than one**. The reference API shows the same signature, 392ms against 407ms, which is the evidence that it shares this design.

On the two sets of latency numbers in this paper. The figures here (69 and 74ms) were taken while a training run shared the GPU; those in §6.5 (37.7 to 55.9ms) were taken under lighter load. Absolute values are therefore not comparable across the two sections and only the within-section ratios are meaningful. We report both rather than the flattering one, and neither is a throughput measurement: everything here is batch size 1, which is the interactive case. Sustained throughput under load is not measured in this work.

4 Data

Since our central claim is that generalization lives in the training data rather than the architecture, the data deserves the same scrutiny usually reserved for the model.

4.1 Training mixture

We assemble 279 tasks and 323,466 rows from `tasksource` [8], normalised into the typed-decision format. Composition after cleaning:

primitive	tasks	notes
choice	234	option counts 2 to 174; 80 tasks have per-example menus
noul	34	
score	11	mostly three-level sentiment; see §8
total	279	323,466 rows

The lopsidedness is a limitation, not a design choice, and it predicts the result pattern: **choice** transfers best and **score** worst.

Per-example option menus. 80 tasks come from the multiple-choice family, where each row carries its own answer set rather than sharing a fixed menu. Supporting this in the trainer is what unlocked that family, which is 261 of the 668 tasks `tasksource` exposes.

4.2 Contamination guard

A held-out suite is worthless if the training mixture contains the same data under a different name, and aliasing is the normal case: `tasksource` calls one dataset `banking77`, while the Hub also carries `PolyAI/banking77` and `mteb/banking77`.

Each evaluation task therefore declares every alias of its source, a manifest holds the union, and a matcher checks full names, owner-stripped basenames, and glob patterns rather than performing a set intersection. The check runs before every training run and refuses to proceed on a hit; it excluded 23 tasks when building this mixture, and caught a real leak during development. We verify it with a positive control, injecting `banking77` into a clean mixture and confirming rejection, since a guard that has never fired is indistinguishable from a guard that cannot.

4.3 Auditing, and what it found

Training is the expensive step, so faults that survive it and surface hours later as a bad number are worth catching first. We audit for gold answers outside their own menu, **score** golds stored as booleans (in Python `True == 1`, which silently rewrites levels 0 and 1), class imbalance too severe to learn, near-duplicate **noul** descriptions, split leakage, and contradictory labels.

Three findings changed the data materially.

The gold answer was always first. `tasksource` ships multiple-choice tasks correct-answer-first. Measured after ingestion, the gold sat at index 0 in **100% of rows across all 83 ingested tasks**. Stored that way, the data teaches “pick the first option” to anything reading the menu as written. We shuffle at ingestion so the stored data is honest, and drop tasks whose gold remains at a fixed index or is a single constant string.

Contradictory labels. 1,994 rows carried the same state under the same menu with different answers; whichever the model predicts, some are wrong, so the gradient is noise. One task contributed 195.

Menus too long to pack. 1,787 rows had option sets exceeding the context budget on their own, some options being multi-paragraph passages.

Finally, we sweep every example through the same dataset object the trainer uses, including augmentation, before training starts: 323,466 examples verified in 227 seconds of CPU. Two earlier runs died on packing errors after twenty minutes and ten thousand steps respectively, and both faults were findable in advance.

4.4 Label-space augmentation

Our mixture’s option counts topped out near 20 while the evaluation suite reaches 151. A model that has never seen a large menu collapses onto one label when handed one, which is what an exact 0.000 looks like: chance alone would have produced roughly three correct in 200.

The intervention is to manufacture the missing shape. With probability π , a training menu is padded with labels borrowed from other tasks in the mixture, sampling a target size log-uniformly so small menus stay common and large ones appear often enough to matter. The gold answer is unchanged and still correct, so the example remains valid, but the model must now discriminate against a large menu.

Option order is shuffled every epoch for a second reason: on the reference system we measured the argmax flipping across option orderings on 7.5% of confusable items, at $2.8\times$ the resampling floor. That is per-item instability rather than a positional prior, so it cannot be corrected post hoc and must be trained out.

4.5 Evaluation suites

Held-out suite (Study G). Seven public tasks, 600 rows each, never trained on: `clinc_oos` ($K=151$), `banking77` ($K=77$), `massive_intent` ($K=60$), `sst5` and `helpsteer` (`score`, $K=5$), `ag_news` ($K=4$), and `civil_comments` (`noul`). The high-cardinality end is the point: a 151-way menu is where models that look fine on four-way classification fail.

Every evaluation reports a *held-in control*: accuracy on tasks the model was trained on. If held-out is at chance and held-in is high, the model genuinely fails to transfer; if both are at chance,

Algorithm 2 Label-space augmentation for one choice question

Require: menu O with gold y , distractor pool D , probability π , cap M

```
1: if UNIFORM(0, 1)  $\geq \pi$  then return  $O$ 
2: end if
3:  $n \leftarrow \lfloor \exp(\text{UNIFORM}(\ln(|O|+1), \ln M)) \rfloor$   $\triangleright$  log-uniform target size
4:  $E \leftarrow \emptyset$ 
5: while  $|O| + |E| < n$  and attempts remain do
6:    $d \sim D$ 
7:   if  $d \notin O$  and  $d \notin E$  then  $\triangleright$  never add a plausibly correct label
8:      $E \leftarrow E \cup \{d\}$ 
9:   end if
10: end while
11: return SHUFFLE( $O \cup E$ )  $\triangleright y$  still present and correct
```

it is a bug. We also run a trivial three-way sanity task that any working harness must pass. This is not ceremony. The sanity check caught a real partially-loaded-model bug that had produced a plausible, and wrong, results table.

Healthcare router (Study S). A synthetic pharmacy routing task: 988 items across 18 tiers, 395 train and 450 test, with ten questions per item (one intent **choice**, five topic **nouls**, four safety gates).

Its first version was useless: the reference API scored a clean 1.000 on four of its tiers, which cannot distinguish any two systems. We rebuilt it targeting where that system actually failed: obliquely-worded clinical risk, three-intent compounds, negation and conditionals, near-miss out-of-scope, subtle prompt injection, and degraded voice-transcript text. Splits are disjoint by content rather than by row, so paraphrases and noisy variants of an item all stay in one split.

5 Experimental setup

5.1 Two studies

The experiments answer two different questions, and they disagree. Keeping them separate is what makes the disagreement legible rather than contradictory.

Study G (general capability). Train on the 279-task mixture, evaluate on seven tasks never trained on. *Does it work on a schema you have no labels for?* This is the harder claim and the reference system’s main advantage.

Study S (specific use case). Train on 395 examples of the healthcare router, evaluate on its 450-item test split. *Given labels for your actual problem, how good does it get?*

5.2 Backbones

	encoder	decoder
model	ModernBERT-base	Qwen3-1.7B
parameters	150M	1,725M
attention	bidirectional	causal + block-diagonal
readout	linear at marker	logit(yes) – logit(no)

“Decoder” names the attention pattern, not the usage: nothing is generated token by token, and inference is a single forward pass with a parallel read at marker positions.

5.3 Training configurations

Study G uses one epoch over the mixture, batch 16 (encoder) or 8 (decoder), context 2048, label-space augmentation at $\pi=0.5$ with cap $M=120$. Study S uses six epochs over 395 examples. Four adaptation regimes are compared in §6.4: encoder full fine-tune, decoder full fine-tune, decoder head-only, and decoder LoRA ($r=16$, $\alpha=32$, on all attention and MLP projections, $\text{lr} = 2 \times 10^{-4}$).

5.4 Metrics

Accuracy, macro-F1, and multi-label exact-set match, with bootstrap confidence intervals over examples. For comparability across tasks with very different K we report *multiple of chance*, $\text{acc} \cdot K$, since raw accuracy is not comparable: 0.28 on a 77-way menu is strong and on a 4-way menu is worse than guessing.

Two metric choices are load-bearing.

AUROC on binary tasks. Argmax accuracy on a `noul` describes the 0.5 threshold as much as the model. A task can rank almost perfectly and still score exactly the class balance by putting all probability mass on one side. We report both, and the gap between them is a calibration result rather than a transfer result.

Chance is not always the right floor. A skewed task can be beaten by always predicting its most common label. Where that baseline exceeds $1/K$ we test against it explicitly, which changes one of our conclusions (§6.2).

Paired comparison. Against the reference system we use exact McNemar on the same items, reporting b_{10} (we are right, they are not) and b_{01} . An aggregate delta cannot distinguish a real difference from two systems disagreeing equally in both directions.

5.5 Reproducibility

All runs are on one H100. The contamination guard runs and logs on every training run. Checkpoints record their backbone, adapter rank, prompt preamble, option template, and fitted temperatures, because the same weights scored under a different prompt are a different system. Artifacts are listed in Appendix B.

6 Results

6.1 The architecture does not generalize on its own

Before any task training, we score the held-out suite using only the output format and a pretrained backbone.

task	K	chance	accuracy	95% CI	$\times$ chance
clinc_oos	151	0.007	0.628	[0.587, 0.672]	94.9
massive_intent	60	0.017	0.473	[0.430, 0.512]	28.4
banking77	77	0.013	0.290	[0.255, 0.323]	22.3
ag_news	4	0.250	0.735	[0.698, 0.772]	2.9
sst5	5	0.200	0.412	[0.373, 0.452]	2.1
civil_comments	2	0.500	0.683	[0.648, 0.718]	1.4
helpsteer	5	0.200	0.262	[0.225, 0.297]	1.3
mean multiple of chance					21.9

Table 1: Held-out transfer, 600 rows per task, never trained on. The suite began this work at $0.9\times$ chance. Every interval excludes its chance floor.

backbone	configuration	held-out mean
encoder, 150M	pretrained masked-LM head at the marker	$0.7\times$ chance
encoder, 395M	same readout, larger model	$2.2\times$ chance
decoder, 1.7B	state and options concatenated plainly	$0.8\times$ chance
decoder, 1.7B	+ each option framed as yes/no	$2.9\times$ chance
decoder, 1.7B	+ a preamble stating the task	$11.5\times$ chance
decoder, 1.7B	<i>control</i> : shortened framing	$5.7\times$ chance

The two halves of the table are different backbones and are not directly comparable; each is internally consistent. Within the encoder rows, a strong bidirectional model with a clever output format scores *below chance* and its larger sibling barely clears it. Within the decoder rows, the format is worth an order of magnitude.

The last row is the control that matters: shortening the framing to `{opt}\ncorrect? answer` roughly halves the mean, so the gain is the format doing work rather than an incidental wording choice.

The same sensitivity appears in option rendering. On the held-out binary task, scoring the two `noul` descriptions rather than bare polarity costs AUROC 0.63 against 0.712, because the descriptions restate one judgement from opposite sides and a model scoring both measures their overlap.

6.2 Held-out transfer

Table 1 gives the final encoder trained on the audited 279-task mixture. Harness sanity 1.000; held-in control 1.1–2.3 $\times$.

One task is weaker than the table implies. Against the majority-class baseline rather than $1/K$, `civil_comments` wins outright (0.683 vs 0.500, $p < 10^{-15}$) but `helpsteer` sits on the line: 0.262 against a 0.233 baseline, $p = 0.050$. Six of seven are unambiguous; the seventh beats chance while only matching the trivial baseline.

6.3 What produced the transfer

Label-space augmentation is the single largest effect we measured:

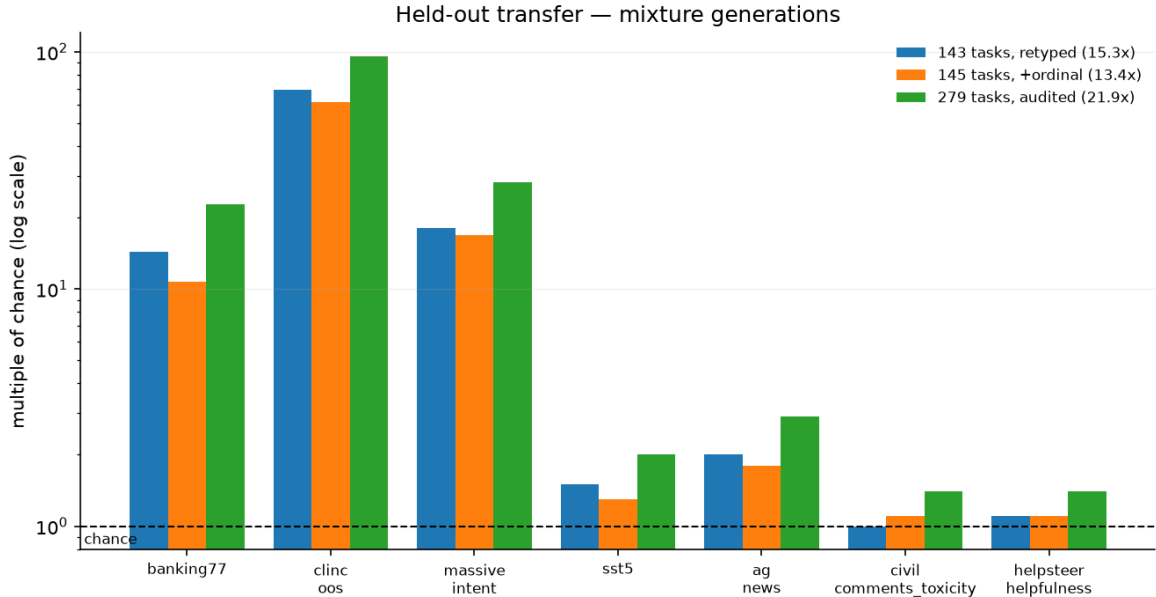


Figure 3: Held-out transfer across three mixture generations, log scale, chance marked. The middle generation is *worse* than the first: that is negative transfer from a mixture change that traded away **choice** tasks, retained here because a plot of only the best run would imply progress that did not happen.

held-out task	K	before	after
banking77	77	0.000	0.205
clinc_oos	151	0.000	0.180
massive_intent	60	0.040	0.305
ag_news	4	0.225	0.655

Two exact zeroes become usable numbers. You do not need datasets with large menus; you can synthesize them. The caveat is that the mixture also grew from 61 to 141 tasks in the same change, so scale and shape are not separated in this pair.

Scaling the audited mixture to 279 tasks took the mean from $17.6\times$ to $21.9\times$ and moved every task (Figure 3).

6.4 Router: a fine-tuned small model against a zero-shot API

The comparison is asymmetric, in our favour, and we state it before the numbers. OpenJev is fine-tuned on 395 labelled examples of this task. The reference API is not: it has never seen this task’s training split, and produces its answers from the schema alone. This is not a like-for-like contest, and a reader should discount it accordingly. What it establishes is narrower than “we beat Jev”: *if you can label a few hundred examples, a 1.7B open model reaches and passes a closed API operating zero-shot on the same task.* That is a claim about the value of labels as much as about either model.

Two rows that look like wins are not. The `compound_3` tier (0.903 against 0.645) and oblique clinical recall (0.966 against 0.793) are both large margins that fail the significance test, at $p = 0.057$

	OpenJev <i>fine-tuned</i>	reference <i>zero-shot</i>	b_{10}	b_{01}	p	winner
intent	0.979	0.941	14	1	9.8e-04	OpenJev
multi-label exact set	0.909	0.822	57	18	7.2e-06	OpenJev
scope gate	0.978	0.880	49	5	3.9e-10	OpenJev
clinical gate	0.987	0.978	7	3	0.34	level
abusive gate	0.993	0.996	2	3	1.00	level
injection gate	0.993	0.996	1	2	1.00	level
compound_3 tier	0.903	0.645	11	3	0.057	level
oblique clinical recall	0.966	0.793	5	0	0.063	level

Table 2: Rank-16 LoRA on Qwen3-1.7B, 395 training examples, 258 seconds, paired on the same 450 test items with exact McNemar. OpenJev is fine-tuned; the reference is zero-shot. Three wins, five ties, no losses. The last two rows have $n=31$ and $n=29$, where even a large margin struggles to reach significance: the oblique-clinical tier floors at $p = 0.0625$ with zero losses.

and $p = 0.063$, because the tiers hold 31 and 29 items. We report them as level. They are the two tiers we most expected to win, which is exactly why they need the test rather than the margin.

compound_3 is nonetheless where the `noul` primitive earns its place in design terms: three simultaneous intents in one message, which a single `choice` softmax cannot express at all, whatever the p -value on 31 items says.

The smallest configuration is the best one.

configuration	intent	trainable	artifact
encoder, all open	0.899	150M	0.6 GB
decoder, head only	0.666	4.2M	17 MB
decoder, all open	0.929	1,725M	3.4 GB
decoder, LoRA $r=16$	0.979	17M	87 MB

Adapting 17M parameters beats opening 1,725M, at 1/39 the artifact. The likely explanation is the learning rate a full fine-tune tolerates: 10^{-5} over six epochs of 395 examples barely moves a 1.7B model, while LoRA at 2×10^{-4} adapts quickly without disturbing the weights it rides on. Neither was tuned beyond one setting, so this is “adapters are the right default here” rather than a tuned optimum.

Intermediate-task transfer. Starting the encoder specialist from a general checkpoint rather than raw weights, with identical data and schedule, moves intent $0.544 \rightarrow 0.899$ and multi-label $0.453 \rightarrow 0.789$ ($p = 6e-18$). Two safety gates that never fired at all from scratch reach 0.857 and 0.583 recall. This is the small-target-data regime where STILTs predicts the largest gain.

But the general checkpoint was not needed here. The 0.979 LoRA used no mixture training at all; it started from base Qwen3-1.7B. The general checkpoint earns its keep on schemas you have no labels for, not on the task you actually care about.

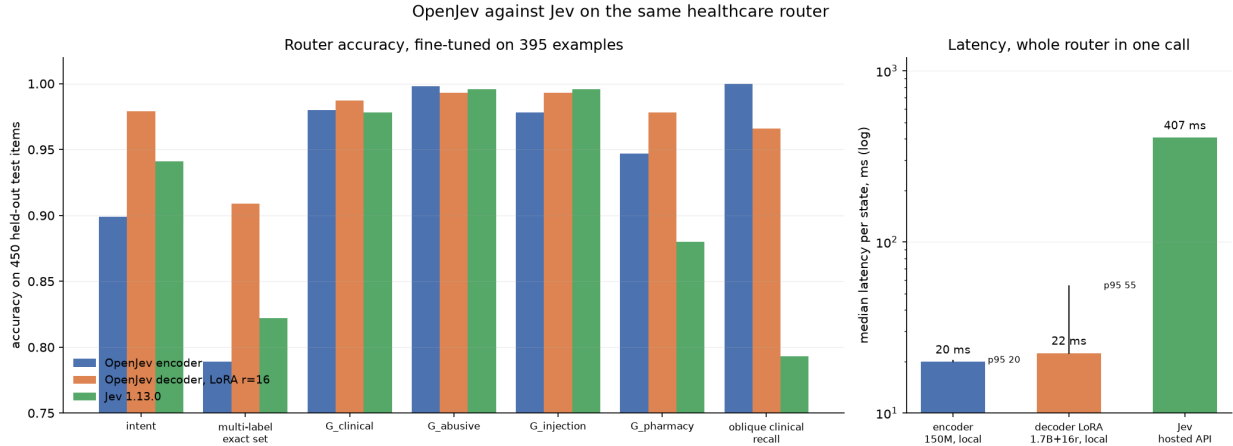


Figure 4: OpenJev against the reference API on the router. Accuracy left, latency right. The latency panel is not like-for-like: our figures are local GPU compute and the reference is an end-to-end hosted call including network, so it is the latency a user experiences rather than a claim about their model’s speed.

6.5 Latency

	p50	p95	vs encoder
encoder, 150M	37.7 ms	50.9 ms	—
decoder LoRA, unmerged	74.0 ms	110.5 ms	1.96×
decoder LoRA, merged	40.4 ms	55.9 ms	1.07×

Measured back to back under identical load; read the ratios rather than the absolute values. An unmerged adapter costs roughly 2× for nothing, since it is an extra matmul per target module at every layer. Merged, a model eleven times larger runs 7% slower.

6.6 Benchmark transfer is not deployment readiness

The general checkpoint scores 21.9× chance on the held-out suite. Applied zero-shot to the routing task:

	zero-shot	fine-tuned	reference, zero-shot
intent	0.601	0.902	0.941
multi-label exact set	0.427	0.789	0.822
clinical gate recall	0.111	0.991	0.926

Intent at 0.601 on a six-way menu is genuinely above chance, so the transfer is real. It is also 34 points behind the reference, and the clinical gate fires on one in nine of the cases it should catch, which is not a gate. Clearing chance on academic benchmarks with clean label sets is a materially weaker claim than working on a deployment task you have no labels for, and the two come apart sharply here. Safety gates in particular appear to require task supervision.

7 Ablations, negative results, and one retraction

Reporting only what worked would misrepresent both the difficulty and the method. Three interventions that sounded right did nothing or actively hurt, and one conclusion we published internally was wrong.

7.1 Interventions that failed

Ordinal scale augmentation: null. The mixture’s **score** tasks are mostly three-level sentiment scales, against a held-out task wanting a five-level judgement. We synthesized the missing diversity the way label-space augmentation synthesized menu size: merging adjacent levels and remapping the gold, and restating a scale in another vocabulary of the same length within its semantic family. Ablated at $\pi_{\text{scale}} \in \{0, 0.5\}$ with everything else fixed:

	$\pi_{\text{scale}}=0$	$\pi_{\text{scale}}=0.5$
helpsteer	0.110 (0.6 $\times$)	0.128 (0.6 $\times$)
sst5	0.342 (1.7 $\times$)	0.348 (1.7 $\times$)
suite mean	16.5$\times$	15.7 $\times$

Both ordinal tasks land on the same multiple of chance either way, and the mean is slightly *worse* with it. The mechanism is correct and tested (gold stays on the right level, merges stay contiguous and monotone) it simply is not the bottleneck.

Adding a primitive by removing another: negative, twice. The mixture was poor in **noul** (13 tasks) and **score**. We recovered 21 real yes/no tasks by recognising negation pairs (**not_paraphrase** vs **paraphrase**) that the builder had been emitting as two-option menus, and separately added two real ordinal datasets. Both made things worse:

mixture	choice tasks	banking77	suite mean
143 tasks, baseline	121	0.278	17.6$\times$
143 tasks, 21 retyped to noul	100	0.187	15.3 $\times$
145 tasks, + 2 ordinal	100	0.138	13.4 $\times$

banking77 tracks the **choice** task count exactly. Five of seven held-out tasks are **choice**, so converting or diluting **choice** tasks is a trade rather than an improvement. This is textbook negative transfer [4], and we caused it twice before measuring it. The fix was scale: the audited 279-task mixture restored **choice** to 234 and added tasks on top, reaching 21.9 $\times$.

7.2 A conclusion we retracted

For several runs we reported that the **noul** primitive does not transfer, on the evidence that the held-out binary task sat at exactly 0.500 accuracy with macro-F1 0.333, which is a model answering the same way every time. We spent two separate interventions on it.

Both were aimed at the wrong thing. The model ranks that task at AUROC 0.714 and reaches 0.680 accuracy at a fitted threshold; it puts 98% of its probability mass above 0.5, so **argmax** calls everything positive and accuracy lands on the class balance. **That is a calibration result, not a transfer result.** Reporting **argmax** accuracy alone on an uncalibrated binary measures the threshold as much as the model, and our evaluation now reports AUROC alongside it.

Two measurement errors of our own contributed, and both are worth stating because they are easy to repeat:

- **A label inversion in the inference helper.** Option names were derived in *dictionary* order, so a `noul` whose criteria were written `{true: ..., false: ...}` had its two probabilities swapped. This turned AUROC 0.712 into 0.288, exactly $1 - 0.712$, and looked like a model ranking toxic comments as cleaner than clean ones. The giveaway was the arithmetic.
- **A partially loaded model that produced a full results table.** A decoder was built without its trained head, so `load_state_dict(strict=False)` silently dropped six tensors and scored the untrained readout, reporting $5.8\times$ chance where the truth was $16.2\times$. A warning printed; a complete results table printed under it. Loaders now raise.

We also credited the held-in control with catching the second bug, and that was wrong too: the decoder reads $1.0\text{--}1.3\times$ on held-in tasks even when it loads perfectly, because a 4.2M head on a frozen backbone barely fits its own training mixture. Only the harness sanity task caught it.

7.3 Interventions that worked

For completeness, the positive results, ordered by effect size: label-space augmentation (§6.3); mixture scale and auditing ($17.6\times \rightarrow 21.9\times$); prompt format on a decoder backbone ($0.8\times \rightarrow 11.5\times$); intermediate-task transfer (+36 points at 395 target examples); LoRA over full fine-tuning (0.979 vs 0.929 at $1/39$ the size); and adapter merging ($1.96\times \rightarrow 1.07\times$ latency).

7.4 Methodological caveat

One choice in this paper was made badly. The `noul` rendering rule (§3) was selected by comparing variants on the held-out task itself, which is the wrong place to choose anything. The change is principled and small, and the three-way spread we observed across connective phrasings (0.25 to 0.75 AUROC) is large enough that it should be re-settled on held-in data before the held-out number is quoted as clean.

8 Discussion

8.1 What the evidence supports

The split is clean and it is about labels. **Given roughly 400 labelled examples for your task, a 1.7B model with an 87MB adapter beats a commercial typed-decision API** on a hardened routing benchmark, runs on your own hardware, returns full-precision probabilities, and is deterministic. **Without labels, it does not come close:** 0.290 on Banking77 against roughly 0.820.

Both halves matter. The first says the capability is not gated behind scale or a vendor. The second says the vendor’s actual product, working on a schema you have never labelled, is a real and unsolved thing.

8.2 When this fits a real problem

Published walkthroughs of the reference API give a useful external check on whether our primitive set matches real demand. Three that circulated widely: filtering 38,000 LinkedIn connections against

an ideal-customer profile, cleaning 4,710 auto-collected Gmail contacts, and triaging six months of arXiv abstracts for relevance to non-technical readers.

All three use only **choice**, **score** and **noul**; all three ask several questions per record in one call; all three threshold on the returned confidence. Menu sizes run from 4 to 15, well inside what we handle. So the abstraction transfers, which is worth stating because we did not design it and it could have turned out narrower than the real demand.

Whether *our* implementation fits depends on one thing:

	no labels	a few hundred labels
schema you have never seen	reference API	OpenJev
cost per record at scale	comparable	OpenJev (local)
need a real probability	OpenJev	OpenJev
need reproducibility	OpenJev	OpenJev

The probability and reproducibility rows are ours regardless of labels: the reference quantises to a 0.01 grid with 71.9% of values at a hard zero, which matters directly for the confidence thresholds all three of those applications expose to the user, and it offers no seed.

Two practical observations. First, on our own evidence a task dominated by **score** questions is the worst case for OpenJev, since the ordinal primitive is where we only match a majority baseline; two of the three walkthroughs contain such questions. Second, and more usefully: each of those three workflows *already* contains a labelling step, a 40-row corrected evaluation file in one and a 50-item checkpoint in another, used for validation rather than training. The labels a fine-tune needs are already being produced. Raising 40 to 300 converts a validation artifact into a training set, and on our measurements that is the difference between trailing the reference API and passing it.

8.3 Why the data mattered more than the model

The most useful finding here is negative: a strong encoder with a clever output format scores below chance untrained. Nothing about scoring options at marker positions confers generalization. What produced it was the composition of the training mixture, and in particular a property we could manufacture, namely menu-size diversity, rather than one we had to find.

This has a practical consequence. Our mixture is 234 **choice** tasks against 34 **noul** and 11 **score**, and the result pattern follows exactly: **choice** transfers, the other two barely do. That is a statement about 279 tasks scraped from one source, not about the architecture. The fastest route to a better general model is a more balanced typed-decision corpus, and assembling one is beyond what a single project can do.

8.4 Limitations

The ordinal primitive is weak. **helpsteer** beats chance but only draws level with predicting the most common label. The mixture has 11 **score** tasks, mostly three-level sentiment. We searched twice for more; ordinal scales with five or more levels, in domains that are not response quality, are scarce in accessible form. We deliberately excluded LLM-response-rating corpora because they are the held-out task in all but name, and training on a near-twin would move the number without moving the ability.

No ordinal or rejection head. **score** is a softmax over levels; we use the ordering only in rendering and in an expectation-based readout. Rank-consistent objectives [7] and adaptive rejection boundaries [12] are the obvious next step and are not implemented.

Zero-shot on a real task is much worse than on benchmarks. 21.9× chance on the held-out suite coexists with intent 0.601 and a clinical-gate recall of 0.111 on the routing task. Safety gates in particular appear to need task supervision, and we would not deploy any zero-shot gate on this evidence.

Synthetic evaluation. The routing task is handwritten and systematically composed, not real traffic. It is evidence a method works, not a claim about any production distribution.

One-day black-box baseline. All reference figures are measurements of a hosted endpoint on a single day from one network location.

Untuned comparisons. The LoRA-versus-full-fine-tune result compares one hyperparameter setting each. We believe the direction and not the magnitude.

Single runs, no seed variance. Every number in this paper is one training run. The confidence intervals are bootstrap over the test set, which captures sampling noise but says nothing about run-to-run variance from initialisation, data order, or augmentation sampling. Differences of one or two points between configurations should not be read as real on this evidence. The large effects (a literal 0.000 becoming 0.205; 0.666 against 0.979) are far outside any plausible seed noise; the small ones are not.

The obvious baseline is missing. We do not compare against m independent single-question classifiers, which is the natural alternative to packing and the first thing a careful engineer would try. Packing’s advantage should be cost rather than accuracy, since separate classifiers have strictly more capacity per question, and our claim is about answering ten questions for the price of one. But we did not measure it, so the accuracy side of that trade is unquantified here. Nor do we compare against a general-purpose LLM with constrained decoding, which is what most teams would reach for today.

Two tiers we expected to win are statistically level. The `compound_3` tier and oblique clinical recall show large margins (0.903 vs 0.645 and 0.966 vs 0.793) at $p = 0.057$ and $p = 0.063$, on 31 and 29 items. They are reported as ties. A larger version of those tiers is the cheapest way to settle whether the margins are real.

8.5 Future work

The experiment that would most change these conclusions is a balanced typed-decision corpus with real `score` and `noul` diversity at the scale Flan suggests. Beyond that: adapter composition, where a task adapter is initialised from a general adapter, which is the parameter-efficient analogue of the +36-point intermediate-task result; rank-consistent ordinal heads; and calibrated rejection for out-of-scope inputs.

8.6 Conclusion

Typed decisions do not need a large model, and they do not need generation. They need the right output format, an attention mask that keeps questions independent, and above all training data whose *shape* matches deployment. Of those three, only the last was hard, and it is the one that transfers least from the literature to practice.

References

- [1] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *ACL*, 2020.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- [3] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. The flan collection: Designing data and methods for effective instruction tuning. In *ICML*, 2023.
- [4] Jason Phang, Thibault Févry, and Samuel R. Bowman. Sentence encoders on STILTs: Supplementary training on intermediate labeled-data tasks. *arXiv preprint arXiv:1811.01088*, 2018.
- [5] Nir Ratner, Yoav Levine, Yonatan Belinkov, Ori Ram, Inbal Magar, Omri Abend, Ehud Karpas, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. Parallel context windows for large language models. In *ACL*, 2023.
- [6] Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, et al. Multitask prompted training enables zero-shot task generalization. In *ICLR*, 2022.
- [7] Xintong Shi, Wenzhi Cao, and Sebastian Raschka. Deep neural networks for rank-consistent ordinal regression based on conditional probabilities. *arXiv preprint arXiv:2111.08851*, 2021.
- [8] Damien Sileo. tasksource: A dataset harmonization framework for streamlined nlp multi-task learning and evaluation. In *LREC-COLING*, 2024.
- [9] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Roberts, Ramesh Nallapati, et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.
- [10] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [11] Ping Yang, Junjie Wang, Ruyi Gan, Xinyu Zhu, Lin Zhang, Ziwei Wu, Xinyu Gao, Jiaying Zhang, and Tetsuya Sakai. Zero-shot learners for natural language understanding via a unified multiple choice perspective. In *Findings of EMNLP*, 2022.
- [12] Hanlei Zhang, Hua Xu, and Ting-En Lin. Deep open intent classification with adaptive decision boundary. In *AAAI*, 2021.

A Terminology for readers new to the area

This appendix defines every term the paper uses, assuming no background. Nothing here is needed by a reader who already works on language models; it exists so the main text can stay short.

A.1 The problem

State

The input being judged: a customer message, a product review, a comment. One state can have many questions asked about it.

Question

One decision to make about the state, with instructions and a set of candidate answers.

Option, menu, criteria

Used interchangeably for the candidate answers of a question. A *fixed menu* is shared by every row of a task; a *per-example menu* differs row by row, as in reading comprehension where each question has its own answers.

K

The number of options. $K=151$ means a 151-way choice.

Primitive

The type of a question: **choice** (pick one of K), **score** (pick one level on an ordered scale), **noul** (yes or no).

Multi-label

When more than one label can be true at once. A single **choice** cannot express this, because its probabilities sum to one across options. Several **noul** questions can.

A.2 How models process text

Token

A chunk of text, roughly a word piece. Models read and write sequences of tokens, not characters.

Embedding, hidden state

The vector a model holds for each token position after processing. The hidden state at a position is what we read a decision from.

Attention

The mechanism by which a token’s representation is built from other tokens. An *attention mask* says which positions may look at which.

Encoder vs decoder

Two attention patterns. An **encoder** is bidirectional: every token sees every other. A **decoder** is causal: a token sees only what came before it, which is what makes text generation possible. **In this paper “decoder” names the attention pattern only.** We never generate text; we run one forward pass and read scores.

Forward pass

One run of the model over an input. Our central efficiency claim is that m questions cost one forward pass, not m .

Logit

A raw, unnormalised score. Logits become probabilities through a softmax.

Softmax

Turns a vector of logits into probabilities that sum to one. Our *grouped* softmax does this separately within each question’s options, because questions in one packed sequence have different K .

Backbone

The pretrained model we build on, before any of our training.

A.3 Training

Pretraining

The expensive first stage, done by others, where a model learns language from large text corpora. We never do this.

Fine-tuning

Further training of a pretrained model on a specific task. Cheap by comparison: 38 seconds in one of our settings.

Epoch

One pass over the training data.

Learning rate

How large a step training takes per update. It matters more than it sounds: our LoRA-beats-full-fine-tuning result is most likely a learning-rate effect (§6.4).

Frozen

A parameter that is not updated during training.

LoRA (Low-Rank Adaptation)

Instead of updating a large weight matrix, train two small matrices alongside it and add their product. The large matrix stays frozen. The **rank** r sets how small; $r=16$ here. The result is an **adapter** of tens of megabytes rather than gigabytes, so one backbone can serve many tasks.

Merging

Folding a trained adapter back into the frozen weights, so inference costs no extra work. An unmerged adapter is roughly twice as slow for no benefit (§6.5).

Multi-task mixture

Training on many tasks at once, so the model learns the general shape of the problem rather than one instance of it.

Intermediate-task transfer

Train on task A, then on target task B. Helps most when B is small. Known in the literature as STILTs [4].

Negative transfer

When adding training data makes the target *worse*. We caused this twice (§7).

A.4 Evaluation

Zero-shot

Evaluating on a task the model was never trained on. The hard claim, and the thing Jev is good at.

Held-out

Data deliberately excluded from training, used to measure zero-shot ability. Worthless if it leaks into training, hence the contamination guard.

Held-in control

Accuracy on tasks the model *was* trained on, reported next to held-out. If held-out is at chance and held-in is high, the model truly fails to transfer; if both are at chance, something is broken. Without this, a bug and a finding look identical.

Chance

What random guessing scores: $1/K$. On a 151-way menu that is 0.007, so an accuracy of 0.29 is strong; on a 4-way menu chance is 0.25 and 0.29 is nearly worthless.

Multiple of chance

Accuracy divided by chance, so tasks with very different K can be compared on one axis.

Majority-class baseline

Always predicting the most common label. On skewed data this beats chance, so it is the stricter floor. One of our results passes the chance test and only ties this one (§6.2).

Accuracy, precision, recall

Accuracy is the fraction correct. *Recall* is the fraction of true positives found, which is what matters for a safety gate. *Precision* is the fraction of positive predictions that were right.

Macro-F1

The harmonic mean of precision and recall, averaged over classes so rare classes count as much as common ones. A binary task scoring macro-F1 0.333 is predicting one class for everything.

AUROC

Ranking quality, independent of any threshold: the probability a random positive is scored above a random negative. 0.5 is chance. It matters because a model can rank well and still score badly at a fixed threshold, which is a calibration problem and not a capability one.

Calibration

Whether a stated probability matches reality. If everything a model calls 0.7 is right 70% of the time, it is calibrated. **ECE** measures the gap; **temperature scaling** is the standard post-hoc fix.

Bootstrap confidence interval

Resample the test set with replacement many times and report the middle 95% of the resulting scores. An interval that excludes chance means the result is unlikely to be luck.

McNemar test

For comparing two systems on the *same* items. Counts only the disagreements: b_{10} where we are right and they are not, b_{01} the reverse, and asks whether the split is lopsided enough to be real. Necessary because two systems can have equal accuracy while disagreeing on a third of the set.

p -value

Informally, the probability of seeing a difference this large if there were no real difference. Below 0.05 is conventionally called significant. With small samples, significance can be unreachable no matter how good the model: one of our tiers has 29 items, so even a perfect score floors at $p = 0.0625$.

A.5 Terms specific to this work

Packing

Writing the state once, then all questions and options after it, as one sequence for one forward pass.

Shared prefix

The state portion of a packed sequence, attended to by every question. The reason m questions are nearly free.

Marker

A token after each option where its score is read.

Block-diagonal attention

The mask that lets each question see the shared prefix and itself, but not other questions, so an answer does not depend on what else was asked.

Label-space augmentation

Padding a training menu with labels borrowed from other tasks. The gold stays correct, but the model must discriminate against a larger menu. Our largest single effect.

Distractor

One of those borrowed labels.

Contamination guard

A check that no evaluation dataset appears in the training mixture under any of its names.

B Released artifacts

All public and Apache 2.0 unless the underlying data says otherwise.

Code. <https://github.com/S1LV3RJ1NX/openjev>

Models. All on the Hugging Face Hub.

artifact	size	link
General encoder checkpoint	0.6 GB	s1lv3rj1nx/openjev-encoder-general
Router LoRA adapter	87 MB	s1lv3rj1nx/openjev-router-lora
Router encoder specialist	0.6 GB	s1lv3rj1nx/openjev-router-healthcare

Datasets.

artifact	size	link
Training mixture	279 tasks, 323,466 rows	s1lv3rj1nx/openjev-mixture
Held-out suite	7 tasks, 4,200 rows	s1lv3rj1nx/openjev-heldout
Healthcare router	988 items, 18 tiers	s1lv3rj1nx/openjev-healthcare-router

Base models (not ours): [answerdotai/ModernBERT-base](#) [9] and [Qwen/Qwen3-1.7B](#) [10]. The mixture is assembled from [tasksources](#) [8].

Loading a model. The adapter is merged into the base weights on load, so no extra inference cost is paid by accident.

```
from openjev import DecisionModel, Choice, Noul
model = DecisionModel.from_pretrained("s1lv3rj1nx/openjev-router-lora")
out = model.answer(state, {"intent": Choice(...), "clinical": Noul(...)})
out["intent"].label, out["intent"].probabilities
```

C Reproducing the main results

result	command
Held-out transfer	<code>scripts/eval_heldout.py --ckpt <ckpt></code>
Router against reference	<code>scripts/eval_router.py --ckpt <ckpt> --dump o.json</code>
Paired McNemar	<code>scripts/compare_to_jev.py --ours o.json --jev j.json</code>
Latency	<code>scripts/bench_latency.py --ckpt <ckpt></code>
Data audit	<code>scripts/audit_data.py <task dir> --deep</code>
Packing check	<code>scripts/check_packing.py <mixture></code>
Binary diagnosis	<code>scripts/diagnose_binary.py --ckpt <ckpt></code>

D Hyperparameters

	Study G (mixture)	Study S (router)
epochs	1	6
batch size	16 enc / 8 dec	8 enc / 4 dec
context	2048	2048 enc / 3072 dec
learning rate	3×10^{-5} enc	3×10^{-5} enc, 2×10^{-4} LoRA
label-space aug. π	0.5, cap 120	off
noul recast π	0.15	off
LoRA	$r=16$, $\alpha=32$, dropout 0.05	same
LoRA targets	<code>q,k,v,o,gate,up,down</code> projections	

Temperature scaling is fitted on a dev split after training and stored with the checkpoint.

E Held-out suite composition

task	primitive	K	source
<code>clinc_oos</code>	choice	151	<code>clinc/clinc_oos</code>
<code>banking77</code>	choice	77	<code>PolyAI/banking77</code>
<code>massive_intent</code>	choice	60	<code>AmazonScience/massive</code>
<code>ag_news</code>	choice	4	<code>fancyzhx/ag_news</code>
<code>sst5</code>	score	5	<code>SetFit/sst5</code>
<code>helpsteer_helpfulness</code>	score	5	<code>nvidia/HelpSteer</code>
<code>civil_comments_toxicity</code>	noul	2	<code>google/civil_comments</code>

Each task declares every alias of its source; the union is the contamination manifest checked before every training run.